Performance Analysis of Secure Container Runtimes on Kubernetes

Mustafa Ocak, Halil İbrahim Kasapoğlu - Advisor: Bahri Atay Özgövde

Boğaziçi University

Repository

Motivation

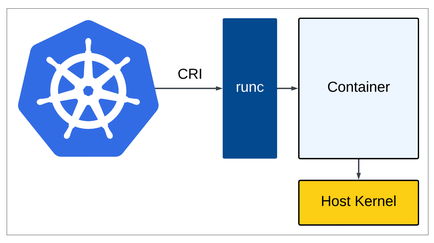
Containerized environments are the backbone of modern cloud-native applications. However, traditional container runtimes like 'runc' prioritize performance over security, leaving room for improvement in isolation and resilience against exploits. This project investigates and benchmarks alternative secure runtimes: 'runsc' (gVisor) and 'Kata Containers', against the widely used 'runc' in a Kubernetes setting.

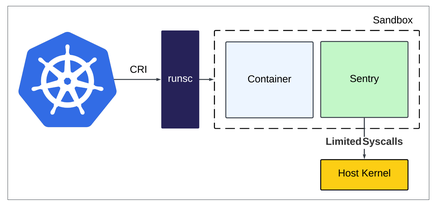
Objective

- Analyze and compare runtime performance under realistic microservice workloads, beyond synthetic call benchmarks.
- Provide a reusable testing suite for replicating and extending performance evaluations.

Runtimes

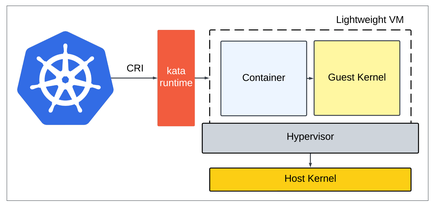
runc is a lightweight runtime adhering to the Open Container Initiative (OCI) standard. It is optimized for performance, making it ideal for high-throughput workloads.



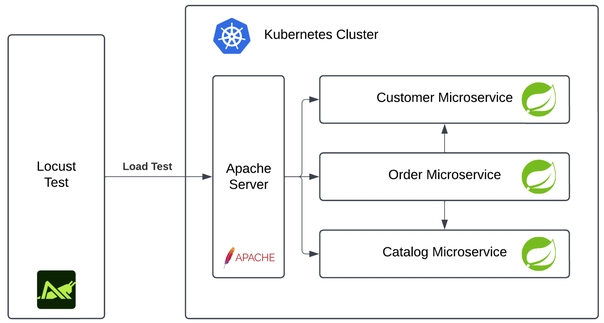


gVisor is a security-focused runtime that uses a user-space kernel to intercept and emulate syscalls, ensuring strong isolation. While it introduces some performance overhead, it offers enhanced security and flexibility for sensitive use cases.

Kata Containers combine containerized performance with the isolation of lightweight virtual machines. It ensures strong security boundaries by running containers inside microVMs.



Methodology



Testing environment

- Microservice stack deployed on Kubernetes cluster with container runtimes swapped.
- Locust for benchmarking analysis. Simulated requests mimicking a microservice user.
- **Performance Metrics Evaluated:**
  - Throughput & Latency
  - Resource usages
  - Startup times
- Prometheus and Kubernetes Metric Server were used for metric evaluation.

Performance Analysis

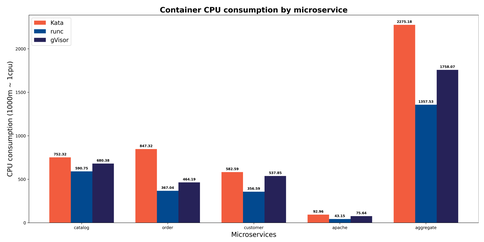


Fig. 1: Average CPU consumption by microservices, aggregated by container runtime during load testing.

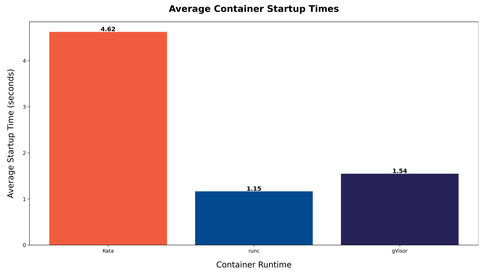


Fig. 2: Average startup times for each runtime class, are calculated by starting multiple replicas of microservice pods and averaging the results.

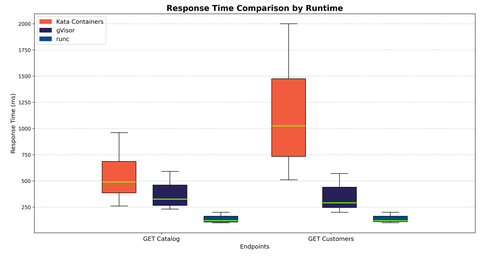


Fig. 3: Response time comparison across container runtimes for Catalog and Customers endpoints, based on Locust benchmark data.

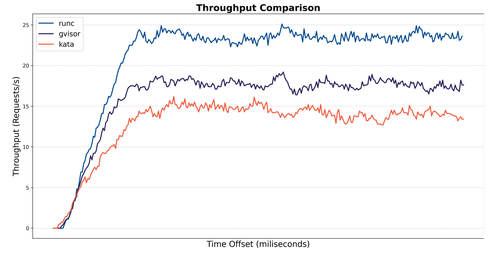


Fig. 4: Average numbers of requests processed per second during peak load testing conducted using Locust

Discussion

- **Low vs High Load Scenarios:** All three runtimes performed similarly in terms of throughput and latency in the low load scenario. Runc maintained consistent performance due to its low resource overhead compared to Kata and gVisor.

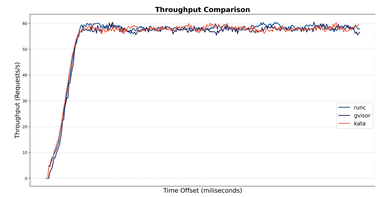


Fig. 5: Low Load Throughput

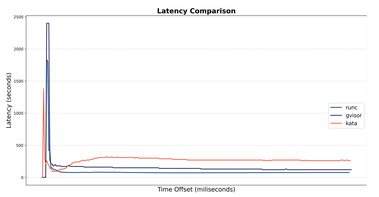


Fig. 6: High Load Latency

- **Startup Times:** Kata displayed significantly slower startup times compared to both runc and runsc. runsc startup times were comparable to runc.
- **Security:** Kata are designed to provide a high level of isolation through VM-based boundaries. runsc offers moderate security by intercepting and sandboxing system calls, reducing the attack surface. runc, being a traditional container runtime, shares the host kernel directly and is more susceptible to exploits involving privileged system calls like `SYS_ADMIN`, which could potentially allow attackers to gain control of the host system.
- **Resource Usage:** runc consumed the least CPU. runsc required slightly more resources due to syscall interception, whereas Kata Containers demanded the most due to VM-based isolation.

Conclusion

The performance of three container runtimes—**runc**, **runsc** (gVisor), and **Kata Containers**—was compared. **runc** demonstrated the best overall performance with minimal overhead, while **runsc** and **Kata Containers** provided enhanced isolation at the expense of higher resource consumption and performance degradation, particularly under high load. The findings indicate that selecting a runtime involves balancing performance and security, depending on the specific workload requirements.

Acknowledgments

We thank Bahri Atay Özgövde for his guidance, Eberhard Wolff for the microservice demo used in our testing, and Ching Ting Leung for the poster template.

References

[1] N. Bhaia, L.-H. Hung, R. Cordingly, and W. Lloyd, "Understanding container isolation: An investigation of performance implications of container runtimes," in *Proceedings of the 9th International Workshop on Container Technologies and Container Clouds*, 2023.

[2] R. Purwoko, D. Priambodo, and A. Prasetyo, "Quantifying of runc, kata and gvisor in kubernates," *ILKOM Jurnal Ilmiah*, vol. 16, no. 1, pp. 12–26, 2024, ISSN: 2548-7779. DOI: 10.33096/ilkom.v16i1.1679.12-26.

[3] E. Wolff, *Microservice kubernetes example*, `https://github.com/ewolff/microservice-kubernetes`, 2023.